Real-Time Process Scheduling

John Wensink

CSC507 Foundations of Operating Systems

Colorado State University – Global Campus

Dr. Joseph Issa

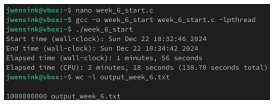
December, 22nd, 2024

Real-Time Process Scheduling

For this week's CT assignment, we were asked to evaluate the large file programs that were created in the previous week and how they would perform if the input sizes were much larger than the initial one million lines of integers. When I had initially completed this assignment, I had done testing with one billion lines of input to give a more accurate picture of how the code performed due to having done the testing on a faster-than-average system designed for gaming and multitasking. This week, I will take some time to further optimize the code using some scheduling concepts that we learned about in week 6 regarding real-time scheduling, number of threads, thread pinning, and niceness/meanness.

Real-Time Scheduling Class

I took the original Linux code for 1 billion lines of code and modified it to experiment with real-time task scheduling to use the ***SCHED_FIFO*** and ***SCHED_RR*** scheduling policies (Grimoire, 2024.) These are true real-time scheduling classes that should give priority to tasks above regular processes. To get started, I copied my code from week 3's Portfolio Milestone project for the multithreaded, I/O optimized code into nano, changed the output file to ***output_week_6.txt***, and saved the code as ***week_6_start.c***. I used ***wc -l output_week_6.txt*** to make sure the output was going to the correct file and that one billion lines of text were written. The processing took 1m56s wall-clock time and 2m18s of CPU time using 4 parallel threads and writing output in binary to a buffer before batch writing the output in chunks to a .txt file.



Now, to get started with adding real-time scheduling optimizations, I used the POSIX ***pthread_setschedparam***, which requires root privileges, so the program will need to run with ***sudo***. To take advantage of ***sched_param***, I needed to include a line at the top to

#include <sched.h>, which allows me to set the scheduling class and priority for the thread. I started out setting the scheduling policy to ***SCHED_FIFO*** by using

if (pthread_setschedparam(pthread_self(), SCHED_FIFO, &param) != 0) (Kerrisk, 2010) (Grimoire, 2024.) This should ensure that the threads receive higher priority than normal processes. The code was compiled and run. Results were about the same as the baseline, saving just two seconds.

```
jwensink@vbox:~$ nano w6_sched.c
jwensink@vbox:~$ gcc -o w6_sched w6_sched.c -lpthread
jwensink@vbox:~$ sudo ./w6_sched
[sudo] password for jwensink:
Start time (wall-clock): Sun Dec 22 10:49:13 2024
End time (wall-clock): Sun Dec 22 10:51:07 2024
Elapsed time (wall-clock): 1 minutes, 54 seconds
Elapsed time (CPU): 2 minutes, 11 seconds (131.21 seconds total)
```

I was expecting a bit better performance, but the VM isn't really running other tasks, so I'm not too surprised by the meager performance gain using FIFO real-time scheduling. For the next step, I changed the scheduling to implement round-robin real-time scheduling by changing the if statement to ***if (pthread_setschedparam(pthread_self(), SCHED_RR, &param) != 0*** (Kerrisk, 2010) (Grimoire, 2024.) This ended up working a bit better than FIFO, saving an additional 2 seconds. Going forward, we will keep the round-robin real-time scheduling, as it is executed in 1m52s of wall-clock time and 2m9s of CPU time. This is a 3.45% improvement over the baseline.

```
jwensink@vbox:~$ nano w6_sched_rr.c
jwensink@vbox:~$ gcc -o w6_sched_rr w6_sched_rr.c -lpthread
jwensink@vbox:~$ sudo ./w6_sched_rr
[sudo] password for jwensink:
Start time (wall-clock): Sun Dec 22 10:56:24 2024
End time (wall-clock): Sun Dec 22 10:58:16 2024
Elapsed time (wall-clock): 1 minutes, 52 seconds
Elapsed time (CPU): 2 minutes, 9 seconds (129.80 seconds total)
```

Thread Counts

From week 3, I learned that performance was maximized using four parallel threads, likely due to my VM being allocated 4 physical CPU cores (8 logical processors.) For this week's assignment, we will continue to experiment with the number of threads using four as our baseline and note the results when breaking up the input into two, five, 10, and 20 files. This is a very easy change to the way this program is set up. I only need to change the line at the top

#define NUM_THREADS at the top for the loop to change the number of lines each process works on. For two threads, we lost three seconds over baseline.

```
jwensink@vbox:~$ nano w6_sched_rr_2threads.c
jwensink@vbox:~$ gcc -o w6_sched_rr_2threads w6_sched_rr_2threads.c -lpthread
jwensink@vbox:~$ sudo ./w6_sched_rr_2threads
[sudo] password for jwensink:
Start time (wall-clock): Sun Dec 22 11:59:43 2024
End time (wall-clock): Sun Dec 22 12:01:38 2024
Elapsed time (wall-clock): 1 minutes, 55 seconds
Elapsed time (CPU): 2 minutes, 3 seconds (123.74 seconds total)
```

For five threads, I would expect to begin to see some benefits when there are more threads than physical cores but less than (or equal to) logical processors (DrMax, 2024.) With five threads, each thread can still be assigned a logical processor, and round-robin scheduling might help distribute execution time more fairly among threads sharing physical resources. It turns out that using five threads made no difference in wall-clock execution time compared to two threads. We are still at three seconds over baseline. However, CPU time went up by 13 seconds compared to two threads and seven seconds over baseline.

```
jwensink@vbox:~$ nano w6_sched_rr_5threads.c
jwensink@vbox:~$ gcc -o w6_sched_rr_5threads w6_sched_rr_5threads.c -lpthread
jwensink@vbox:~$ sudo ./w6_sched_rr_5threads
Start time (wall-clock): Sun Dec 22 12:04:34 2024
End time (wall-clock): Sun Dec 22 12:06:29 2024
Elapsed time (wall-clock): 1 minutes, 55 seconds
Elapsed time (CPU): 2 minutes, 16 seconds (136.94 seconds total)
```

Testing with 10 threads, we will start to see whether the benefit of round-robin scheduling is worth the tradeoff of increased context switching and resource contention. I would expect performance to decrease with 10 threads and 8 logical processors as the frequency of context switching increases and hyperthreading bottlenecks start to appear. As expected, performance decreased with 1m59s wall-clock execution and, much worse, 2m48s CPU time.

```
jwensink@vbox:~$ nano w6_sched_rr_10threads.c
jwensink@vbox:~$ gcc -o w6_sched_rr_10threads w6_sched_rr_10threads.c -lpthread
jwensink@vbox:~$ sudo ./w6_sched_rr_10threads
[sudo] password for jwensink:
Start time (wall-clock): Sun Dec 22 12:16:17 2024
End time (wall-clock): Sun Dec 22 12:18:16 2024
Elapsed time (wall-clock): 1 minutes, 59 seconds
Elapsed time (CPU): 2 minutes, 48 seconds (168.43 seconds total)
```

Testing with 20 threads, I would expect performance to be even worse than 10 threads due to the same reasons of context switching, resource contention, and memory flushing. As expected, the program took 2m2s wall-clock time, and the worst yet, 3m16s CPU time. This is a full 10 seconds worse performance compared with the previously optimized four-thread program.

```
jwensink@vbox:~$ nano w6_sched_rr_20threads.c
jwensink@vbox:~$ gcc -o w6_sched_rr_20threads w6_sched_rr_20threads.c -lpthread
jwensink@vbox:~$ sudo ./w6_sched_rr_20threads
Start time (wall-clock): Sun Dec 22 12:20:57 2024
End time (wall-clock): Sun Dec 22 12:22:59 2024
Elapsed time (wall-clock): 2 minutes, 2 seconds
Elapsed time (CPU): 3 minutes, 16 seconds (196.79 seconds total)
```

With four physical cores, it makes sense that four threads would be the sweet spot for this workload. Each thread can be assigned a core without having to compete for resources. This avoids the contention caused by hyperthreading when more threads than physical cores are used. Each thread benefits from its own L1 and L2 caches, and more threads increase the likelihood of

cache thrashing on the shared L3 (AMD, n.d.) Context switching disrupts cache utilization and adds scheduling overhead, especially for an I/O-heavy program like this one. As the number of threads increases, the overhead of thread management, synchronization, and resource contention appear to outweigh the benefits of additional parallelism. I think the next logical step, using four threads and round-robin scheduling, would be to start experimenting with CPU pinning.

CPU Thread Pinning

In order to minimize context switching, I thought it might be interesting to pin each of the threads to their corresponding CPU. Once pinned, the thread will always execute on that specific core unless explicitly moved (Gandham & Alapati, n.d.) This should take advantage of cache locality by having a thread stay on its same core. The thread can reuse data in the L1 and L2 cache. The first try ended up losing performance over week 3's optimization, but I was curious to see if running it again would produce better results, but it only improved the initial attempt by two seconds.

```
jwensink@vbox:~$ nano w6_sched_rr_4threads_pin.c
jwensink@vbox:~$ gcc -std=c99 -o w6_sched_rr_4threads_pin w6_sched_rr_4threads_pin.c -lpthread
jwensink@vbox:~$ sudo ./w6_sched_rr_4threads_pin
[sudo] password for jwensink:
Start time (wall-clock): Sun Dec 22 12:45:06 2024
End time (wall-clock): Sun Dec 22 12:47:12 2024
Elapsed time (wall-clock): 2 minutes, 6 seconds
Elapsed time (CPU): 2 minutes, 43 seconds (163.93 seconds total)
jwensink@vbox:~$ sudo ./w6_sched_rr_4threads_pin
Start time (wall-clock): Sun Dec 22 12:49:29 2024
End time (wall-clock): Sun Dec 22 12:51:33 2024
Elapsed time (wall-clock): 2 minutes, 4 seconds
Elapsed time (CPU): 2 minutes, 39 seconds (159.96 seconds total)
jwensink@vbox:~$
```

I think that Linux's scheduler is likely taking advantage of efficiencies in load balancing here and that pinning a thread to a CPU is preventing the system from making use of the scheduler on an

I/O-bound program. If the program was strictly CPU-bound, I would expect that thread pinning might provide more of a benefit.

Niceness/Meanness

At this point, I will start testing out other scheduling techniques and remove the real-time process priorities. To start out with a nice value of 0, we will go back to week 3's code and add the lines:

if (setpriority(PRIO_PROCESS, 0, <niceness_value>) != 0) {

perror("Failed to set niceness");

return 1;

(Grimoire, 2024)

With a nice value of 0, the code took exactly the same amount of time as the baseline case.

```
jwensink@vbox:~$ nano w6_nice_0.c
jwensink@vbox:~$ gcc -o w6_nice_0 w6_nice_0.c -lpthread
jwensink@vbox:~$ ./w6_nice_0
Start time (wall-clock): Sun Dec 22 13:04:50 2024
End time (wall-clock): Sun Dec 22 13:06:44 2024
Elapsed time (wall-clock): 1 minutes, 54 seconds
Elapsed time (CPU): 2 minutes, 17 seconds (137.65 seconds total)
```

This was the expected result, as the program's priority should default to 0 when not otherwise specified. To continue testing, I set the nice value to -5. With a nice value range from -20 (highest priority) to +19 (lowest priority), -5 is a moderately aggressive setting (GeeksforGeeks, n.d.), and I expect a quicker time to process. This, however, was not the case. The program, again, executed at the baseline 1m54s. I don't have any background user-level programs running on the VM to isolate the variables when testing. Only the OS-related processes are ongoing. I

wanted to see what would happen if I set the nice value all the way to -20, considering that it might end up making the OS sluggish or even end up crashing the OS. The program will get the most CPU time compared to any other process and potentially starve any lower-priority processes like background tasks, system daemons, or even the desktop environment. I was disappointed to see that the program actually took two seconds longer, even at the maximum nice value.

```
jwensink@vbox:~$ nano w6_nice_minus20.c
jwensink@vbox:~$ gcc -o w6_nice_minus20 w6_nice_minus20.c -lpthread
jwensink@vbox:~$ sudo ./w6_nice_minus20
[sudo] password for jwensink:
Start time (wall-clock): Sun Dec 22 13:16:49 2024
End time (wall-clock): Sun Dec 22 13:18:45 2024
Elapsed time (wall-clock): 1 minutes, 56 seconds
Elapsed time (CPU): 2 minutes, 19 seconds (139.80 seconds total)
jwensink@vbox:~$
```

I think what we have seen so far is that this program is I/O-bound and not CPU-bound.

Conclusion

This week's experiments shed some light on a variety of scheduling optimization tools in the C programming language run in a Linux VM environment. While none of the techniques made a substantial improvement in wall-clock execution times compared to the baseline, the activity highlighted the limitations of the techniques on an I/O bound workload. Real-time round-robin scheduling ended up giving the biggest and only benefit compared to week 3's program, saving 3.45% in execution time over the baseline. The gains were less than anticipated, likely due to the lack of competing system processes in the VM environment. Experiments with varying thread counts lined up with what was learned in week 3, that four threads were optimal, and increasing the number of threads caused performance degradation due to resource contention, context switching, and cache thrashing. Thread pinning, while theoretically

advantageous for CPU-bound workloads (due to cache locality), did not improve performance for this program and even resulted in slower execution times. This reinforces the idea that Linux's default scheduler is already pretty effective for I/O heavy workloads. Adjusting the program's priority with niceness values further confirmed that the workload is I/O bound rather than CPU-bound. Even at the most aggressive priority level (niceness = -20), performance gains were absent, if not marginally worse. These results emphasize the importance of understanding workload characteristics when applying optimizations. For an I/O-bound program like this one, I would be more interested in focusing on disk throughput, buffer sizes, chunk sizes, and efficient file handling rather than CPU scheduling-based optimizations. I have a feeling that next week, when we focus on I/O, I will have an opportunity to get better results when I apply the week's lessons to the program.

References

AMD. (n.d.). AMD Ryzen™ 7 5800X Desktop Processor.

Retrieved December 22, 2024, from:

https://www.amd.com/en/products/processors/desktops/ryzen/5000-series/amd-ryzen-7-5800x.html

DrMax. (2024, March 18). How to determine the number of threads to use in a server. Baeldung. Retrieved December 22, 2024, from:

https://www.baeldung.com/cs/servers-threads-number

Gandham, B., & Alapati, P. (n.d.). OCC Pinning: Optimizing concurrent computations through thread pinning [Presentation slides]. Mahindra University, École Centrale School of Engineering. Retrieved December 22, 2024, from:

https://www.cse.chalmers.se/~elad/ApPLIED2024/slides/Gandham.pdf

GeeksforGeeks. (n.d.). Nice and renice command in Linux with examples. Retrieved December 22, 2024, from

https://www.geeksforgeeks.org/nice-and-renice-command-in-linux-with-examples/

Grimoire. (2024). OpenAI. Chat about Linux process scheduling and optimizations. Retrieved December 22, 2024, from https://chat.openai.com/

Kerrisk, M. (2010). The Linux programming interface (p. 6.9.1). No Starch Press.

Retrieved December 22, 2024, from:

https://www.man7.org/linux/man-pages/man3/pthread_setschedparam.3.html